Rufus Browser Demo

Runs three Rufus Python workflows natively in the browser — no backend, no Node.js, no cloud. Pyodide executes the exact same `WorkflowBuilder` + `Workflow` step functions that run on a POS terminal or a cloud worker. WebGPU-accelerated AI inference (Transformers.js) powers the third workflow via a Python → JS FFI bridge.

Quick start

```
# From the repo root (wheels must be served from there)
python -m http.server 8080

# Open in Chrome (best WebGPU support) or Firefox
open http://localhost:8080/examples/browser_demo/
```

First load: ~8–15 s (Pyodide runtime + pure-Python packages downloaded from CDN). Workflow 3 first run: additional ~5–10 s (23 MB model download, cached by the browser after that).

Files

```
examples/browser_demo/
├── index.html   - Dark-themed SPA: three workflow cards, step
└── worker.js    - ES-module Web Worker: Pyodide + Transformer
```

No existing SDK files are modified.

Architecture

```
index.html  (main thread — UI only)
  |  new Worker("./worker.js", { type: "module" })
  |  postMessage({ type: "run_workflow", workflowType, data })
  |◄ postMessage({ type: "step_done" | "workflow_done" | ... }
  |
  └─ worker.js  (ES-module Web Worker)
       ├─ Pyodide v0.26.x  (Python 3.12 runtime, WASM)
       |    ├─ rufus-sdk 0.8.0 wheel  (served from /dist/)
       |    ├─ InMemoryPersistence    (no SQLite, no disk I/O)
       |    ├─ BrowserSyncExecutor   (SyncExecutor subclass, r
       |    ├─ BrowserObserver       (WorkflowObserver → js FF
       |    └─ WorkflowBuilder + 3 inline Workflow definitions
       |
       └─ Transformers.js v3  (WebGPU / WASM ML — loaded dynam
            └─ Xenova/all-MiniLM-L6-v2  (384-dim sentence embe
```

Why bypass `RufusEdgeAgent`

`RufusEdgeAgent` starts heartbeat loops, SAF sync, and cloud polling — none of which exist without a backend. The demo drives the SDK directly: `Workflow.next_step()` in a loop inside `pyodide.runPythonAsync()`.

Worker init sequence

1. Dynamic-import Pyodide `.mjs` from CDN (tries v0.27.0 → v0.26.4 → v0.26.3 → v0.26.2)
2. Dynamic-import Transformers.js from CDN (tries `@huggingface/transformers@3` → `@xenova/transformers@2`)
3. `pyodide.loadPackage("micropip")`
4. Mock native-extension packages (`cryptography`, `orjson`, `uvloop`, `asyncpg`) via `micropip.add_mock_package` so dependency resolution succeeds without WASM wheels
5. `micropip.install(rufus_sdk_wheel, keep_going=True)` — resolves pure-Python deps from PyPI
6. `pyodide.runPythonAsync(PYTHON_SETUP)` — defines all models, steps, providers, builder
7. `postMessage({ type: "ready" })` — UI enables Run buttons

Three demo workflows

Workflow 1 — Order Fulfillment

Step types: STANDARD + PARALLEL + `WorkflowJumpDirective`

```
ValidateOrder → CheckInventory ──┬── [in stock]  → ProcessPaym
                (PARALLEL:       └── [out stock] → BackorderN
                 check_uk +
                 check_eu)
               (FulfillmentDecision raises WorkflowJumpDirecti
```

State: `order_id`, `items[]`, `total`, `stock_uk`, `stock_eu`, `payment_ref`, `status`, `fulfillment_path`

Workflow 2 — IoT Sensor Pipeline

Step types: STANDARD steps with inline loop logic + `WorkflowJumpDirective`

```
InitPipeline → CollectSensorData → ProcessReadings → ComputeSt
                                   (iterates over     (mean, s
                                    readings list)     anomaly
```

State: `device_id`, `readings[]`, `processed[]`, `anomalies[]`, `mean`, `stddev`, `health_status`, `alert_sent`

Workflow 3 — Transaction Risk Scoring

Step types: STANDARD + async JS FFI (WebGPU embedding) + `WorkflowJumpDirective`

```
ExtractFeatures → GPUEmbedding → ComputeRiskScore → ScoreDecis
                  (calls          (cosine similarity  ├── appr
                   js.runWebGPUInference)  vs 3 risk   ├── mar
                                   patterns)           └── dec
```

State: `txn_id`, `amount`, `merchant_category`, `location`, `feature_text`, `embedding[]`, `risk_score`, `decision`, `explanation`, `inference_ms`, `device_used`

Risk patterns (computed via the same model at first run):

- `"high value wire transfer crypto exchange after midnight"` → HIGH
- `"small grocery purchase coffee shop weekday"` → LOW
- `"online subscription streaming service"` → LOW

Key implementation details

BrowserSyncExecutor

SyncExecutor normally creates a ThreadPoolExecutor in initialize(). Pyodide does not support real threads (no SharedArrayBuffer + COOP/COEP headers in a plain HTTP server).

Override:

```
class BrowserSyncExecutor(SyncExecutor):
    async def initialize(self, engine):
        self._engine = engine
        self._thread_pool_executor = None   # no threads
        self._loop = asyncio.get_event_loop()

    async def dispatch_parallel_tasks(self, tasks, state_data,
        # Run tasks sequentially; merge results as if parallel
        results = {}
        for task in tasks:
            func = _BROWSER_FUNCS[task.func_path]   # looked u
            r = await func(state_data, ctx)
            results.update(r)
        return {"_async_dispatch": False, "_sync_parallel_resu
```

PARALLEL tasks receive state_data as a plain dict (from model_dump()), not as a Pydantic model. Their functions are registered at setup time in _BROWSER_FUNCS under a pseudo-path key (e.g. "__br__.check_warehouse_uk").

BrowserObserver

Bridges Python workflow events to JS via Pyodide's from js import ... FFI:

```
async def on_step_executed(self, wf_id, step_name, step_index,
    from js import notifyStepDone
    notifyStepDone(step_name, json.dumps(result or {}))

async def on_workflow_completed(self, wf_id, wf_type, state):
    from js import notifyWorkflowDone
    notifyWorkflowDone(json.dumps(state.model_dump()))
```

notifyStepDone and notifyWorkflowDone are registered as globalThis functions in worker.js and forward to self.postMessage(...) back to the main thread.

WebGPU / Transformers.js bridge

`globalThis.runWebGPUInference` is registered in `worker.js`:

```
globalThis.runWebGPUInference = async (text) => {
    if (!_extractor) {
        _extractor = await _pipeline("feature-extraction",
            "Xenova/all-MiniLM-L6-v2", { device: _gpuDevice })
    }
    const out = await _extractor(text, { pooling: "mean", norm
    return { embedding: out.data, latency_ms: ..., device_usec
};
```

Called from Python via Pyodide FFI:

```
async def gpu_embedding(state, context, **_):
    from js import runWebGPUInference
    result = await runWebGPUInference(state.feature_text)
    return {
        "embedding": list(result.embedding.to_py()),
        "inference_ms": result.latency_ms,
        "device_used": result.device_used,
    }
```

`navigator.gpu` is available inside Web Workers on Chrome 113+. `_gpuDevice` is set to `"webgpu"` if WebGPU is present, `"wasm"` otherwise — Transformers.js handles the fallback transparently.

Workflow run loop

`WorkflowJumpDirective` returns control to the caller without advancing, so a simple `automate_next=True` chain is not enough for branching workflows. The run loop is:

```
async def run_workflow(wf_type, data_json):
    wf = await _make_workflow(...)
    for _ in range(50):
        if wf.status != "ACTIVE":
            break
        await wf.next_step(user_input={})
```

Each `next_step()` call either advances (and recurses via `automate_next`) or returns after a jump, letting the loop continue from the new position.

Terminal branch guard pattern

Alternate-path terminal steps (e.g. `BackorderNotice`, `SendAlert`) sit at the end of the step list and would execute on the normal path once the main path reaches the end of its steps. A guard at the top of each such function prevents this:

```
def backorder_notice(state, context, **_):
    if state.fulfillment_path in ("UK", "EU"):
        return {}   # normal path: no-op
    return {"status": "BACKORDERED", "fulfillment_path": "BACK
```

micropip dependency management

`rufus-sdk` declares dependencies on `cryptography`, `orjson`, `uvloop`, and `asyncpg` which have no pure-Python WASM wheels. Rather than using `deps=False` (which causes a micropip internal error in Pyodide 0.26.x) or `keep_going=True` (which still raises on unresolvable transitive deps), the correct approach is:

```
for _pkg, _ver in [("cryptography","41.0.0"), ("orjson","3.9.0
                   ("uvloop","0.19.0"), ("asyncpg","0.29.0")]:
    micropip.add_mock_package(_pkg, _ver)

await micropip.install(wheel_url, keep_going=True)
```

Additionally, `rufus/__init__.py` tries to call `uvloop.EventLoopPolicy()` at import time. Setting `RUFUS_USE_UVLOOP=false` and `RUFUS_USE_ORJSON=false` before importing rufus prevents this:

```
import os
os.environ["RUFUS_USE_UVLOOP"] = "false"
os.environ["RUFUS_USE_ORJSON"] = "false"
# now safe to: from rufus.workflow import Workflow
```

Python f-strings inside JS template literals

Python f-strings that contain `${...}` (e.g. `f"Amount: ${amount:.2f}"`) will break a JavaScript template literal because JS interprets `${` as an interpolation start. Any `$` that immediately precedes a Python `{expression}` must be removed or rewritten:

```
# BAD  – JS parse error when this string is inside a JS templ
f"Transaction: ${state.amount:.2f} USD"

# GOOD
f"Transaction: {state.amount:.2f} USD"
```

Console output reference

Message	Source	Normal?
`WebSocket ws://localhost:8081/ failed`	Browser live-reload extension	Yes — ignore
`favicon.ico 404`	Missing icon	Harmless
`Loading micropip, pydantic… / Loaded …`	Pyodide package loader	Expected
`dtype not specified… Using fp32`	Transformers.js	Informational
`Some nodes were not assigned to preferred EP`	ONNX Runtime WebGPU	Normal for first session

Troubleshooting

`micropip` install fails with `alembic` error → `deps=False` was used; switch to `add_mock_package` approach (see above).

`AttributeError: module 'uvloop' has no attribute 'EventLoopPolicy'` → Set `RUFUS_USE_UVLOOP=false` before importing rufus (see above).

`SyntaxError: Missing } in template expression` in worker.js → A Python f-string inside the PYTHON_SETUP template literal contains `${`. Remove the `$`.

Worker fires `onerror` with `message: undefined` immediately → JS syntax error in `worker.js` — run `node --input-type=module < worker.js` to find it.

Wheels 404 → The HTTP server must be started from the **repo root**, not from `examples/browser_demo/`:

```
cd /path/to/rufus
python -m http.server 8080   # correct
```

The wheel is served at `http://localhost:8080/dist/rufus_sdk-0.8.0-py3-none-any.whl`.